Лабораторная работа №4

«Классы и структуры: пиксели и изображения»

Цель: на практике разобраться, чем **структура** (`struct`, значимый тип) отличается от **класса** (`class`, ссылочный тип): что происходит при присваивании и передаче в метод, где и сколько памяти занимают данные, чем отличаются значения по умолчанию и как работает сравнение. Полученные знания применить при написании нескольких фильтров для изображений.

Что сдаём:

1. Проект с кодом всех заданий.
2. Заполненную **таблицу наблюдений** (см. Часть 2) — можно прямо в комментариях к коду, в `.md` или на листочке.
3. Ответы на контрольные вопросы (файл «Лабораторная работа 4 - контрольные вопросы»).

Теория по всем пунктам — в файле «Лабораторная работа 4 - Справочный материал».

Подготовительный этап

1. Создайте консольное приложение.
2. Добавьте в проект файл `Image.cs` из курса ЭИОС («Добавить» → «Существующий элемент», см. справочный материал) и поменяйте в нём пространство имён на своё.
3. Файл `Pixel.cs` из ЭИОС **не берите** — структуру `Pixel` в этой работе вы пишете сами.

Класс `Image` (готовый, его внутренности загрузки/сохранения BMP разбирать не нужно):

- `public Image(string filePath)` — конструктор, загружает картинку из файла.
- `public void Save(string filePath)` — сохраняет картинку в файл.
- `public int Width`, `public int Height` — размеры в пикселях.
- `public Pixel this[int x, int y]` — индексатор: **возвращает копию** пикселя / записывает пиксель целиком.
- `public ref Pixel GetPixel(int x, int y)` — возвращает **ссылку** на пиксель внутри изображения.

Внутри `Image` пиксели хранятся в приватном поле `private Pixel[,] _pixels;`.

Часть 1. Пишем свои типы

Задание 1. Структура `Pixel`

Создайте файл `Pixel.cs` со структурой `Pixel`:

1. **Поля:** `public byte R, G, B;`
2. **Конструктор:** `public Pixel(byte r, byte g, byte b)` — заполняет все три поля.
3. **Вычисляемое свойство (только чтение):** `public byte Brightness` — яркость пикселя по формуле `0.3 * R + 0.59 * G + 0.11 * B`.
4. **Статический метод:** `public static byte Clamp(int value)` — «обрезает» число до диапазона 0–255 и возвращает `byte`.
5. **Статический метод-«фабрика»:** `public static Pixel Gray(byte value)` — возвращает серый пиксель (`R = G = B = value`).
6. **Метод:** `public override string ToString()` — возвращает строку вида `(10, 20, 30)`.

> Подумайте: почему `Clamp` и `Gray` — статические, а `Brightness` — нет? (Это контрольный вопрос.)

Задание 2. Класс-двойник `PixelClass`

Создайте файл `PixelClass.cs` и **скопируйте** в него всё содержимое структуры `Pixel`, заменив `struct` на `class` и имя на `PixelClass` (и имя конструктора тоже). У метода `Gray` тип результата тоже поменяйте на `PixelClass`.

> Операторы `==` и `!=`, которые вы добавите в `Pixel` в Эксперименте 4, в `PixelClass` **не копируйте** — иначе сравнение классов перестанет показывать то, ради чего оно затевалось.

Да, это осознанное дублирование: два типа с **одинаковым** кодом и **разным** поведением — главный «лабораторный стенд» этой работы.

Часть 2. Эксперименты

Создайте статический класс `Experiments` (файл `Experiments.cs`). Каждый эксперимент — отдельный статический метод `Experiment1()`, `Experiment2()`, …, который вызывается из `Main`.

Правило экспериментов: перед запуском каждого эксперимента **сначала запишите предсказание**, что выведет программа, и только потом запускайте. Результаты оформите в таблицу наблюдений:

№	Что делаем	Предсказание	Факт	Почему так
1.1	`Pixel b = a; b.R = 99;` → чему равно `a.R`?			
…				

Неправильное предсказание — это нормально. Ненормально — не суметь объяснить, почему факт такой.

Эксперимент 1. Присваивание

```
public static void Experiment1()
{
    Pixel a = new Pixel(10, 20, 30);
    Pixel b = a;
    b.R = 99;
    Console.WriteLine($"struct: a = {a}, b = {b}");

    PixelClass c = new PixelClass(10, 20, 30);
    PixelClass d = c;
    d.R = 99;
    Console.WriteLine($"class:  c = {c}, d = {d}");
}
```

Эксперимент 2. Передача в метод

Реализуйте в классе `Experiments` четыре метода и вызовите каждый для «свежего» пикселя, выводя пиксель до и после вызова:

```
static void MakeWhite(Pixel p)          { p.R = 255; p.G = 255; p.B = 255; }
static void MakeWhite(PixelClass p)     { p.R = 255; p.G = 255; p.B = 255; }
static void MakeWhiteRef(ref Pixel p)   { p.R = 255; p.G = 255; p.B = 255; }
static void ReplaceWithWhite(PixelClass p) { p = new PixelClass(255, 255, 255); }
```

Для каждого из четырёх вызовов: изменился ли пиксель снаружи метода? Обратите внимание на последний метод: объект — класс, но снаружи ничего не поменяется. Почему?

Эксперимент 3. Массивы: значения по умолчанию и память

Оформите пункты 3.1 и 3.2 отдельными методами (`Experiment3_1()` и `Experiment3_2()`): в обоих есть переменная `structs`, и в одном методе её нельзя объявить дважды.

3.1. `default` против `null`.

```
Pixel[] structs = new Pixel[3];
PixelClass[] objects = new PixelClass[3];

Console.WriteLine(structs[0]);               // что выведет?
Console.WriteLine(objects[0] == null);       // что выведет?

structs[0].R = 5;                            // сработает?
// objects[0].R = 5;                         // раскомментируйте, запустите,
запишите, что произошло, и закомментируйте обратно
```

3.2. Сколько это стоит. Измерьте, сколько байт памяти выделяется в каждом случае. Для этого используйте `GC.GetAllocatedBytesForCurrentThread()` — метод возвращает, сколько байт **всего** выделил текущий поток с момента запуска программы. Разница «после − до» = сколько выделила ваша операция.

```
const int N = 1_000_000;

long before = GC.GetAllocatedBytesForCurrentThread();
Pixel[] structs = new Pixel[N];
long after = GC.GetAllocatedBytesForCurrentThread();
Console.WriteLine($"Pixel[{N}]: {after - before} байт");
```

Сделайте **три** замера и внесите их в таблицу:

1. `new Pixel[N]`
2. `new PixelClass[N]` (только массив)
3. Заполнение массива из п.2: `for (...) objects[i] = new PixelClass(0, 0, 0);`

Ответьте:

- Сколько байт приходится на один элемент в каждом случае? Сравните с суммой размеров полей `R`, `G`, `B`.
- Сколько отдельных объектов появилось в куче в случае `Pixel[]` и в случае `PixelClass[]`?

Эксперимент 4. Сравнение

```
Pixel p1 = new Pixel(1, 2, 3);
Pixel p2 = new Pixel(1, 2, 3);
Console.WriteLine(p1.Equals(p2));
// Console.WriteLine(p1 == p2);   // раскомментируйте: что скажет компилятор?
Запишите код ошибки.
```

```
PixelClass c1 = new PixelClass(1, 2, 3);
PixelClass c2 = new PixelClass(1, 2, 3);
PixelClass c3 = c1;
Console.WriteLine(c1 == c2);
Console.WriteLine(c1.Equals(c2));
Console.WriteLine(c1 == c3);
```

Доработка `Pixel`: добавьте в структуру операторы `==` и `!=`, а также переопределите `Equals(object)` и `GetHashCode()` (см. «Сравнение» в справочном материале). После этого строка `p1 == p2` должна компилироваться и выводить `True`.

Эксперимент 5. Ловушки копирования

Напишите метод и попробуйте скомпилировать **каждую строку по отдельности**. Для каждой строки запишите: компилируется или нет, код ошибки, и почему.

```
public static void Experiment5(Image image)
{
    Pixel[] pixels = new Pixel[3];

    pixels[0].R = 255;                      // (a)
    image[0, 0].R = 255;                    // (b)
    foreach (Pixel p in pixels) p.R = 0;    // (c)
    image.GetPixel(0, 0).R = 255;           // (d)
}
```

Затем найдите **два** способа всё-таки изменить канал `R` пикселя `(0, 0)` в изображении и запишите их.

Часть 3. Класс `Image` изнутри: копия или ссылка?

Задание 3. Метод `Clone()`

Строка `Image copy = image;` **не создаёт** копию картинки (почему — вы уже знаете из Эксперимента 1). Добавьте в класс `Image`:

1. Приватный конструктор `private Image(Pixel[,] pixels)`, который просто сохраняет переданный массив в поле `_pixels`.
2. Публичный метод `public Image Clone()`, который создаёт **независимую** копию изображения. Подсказка: у массива есть метод `Clone()`, возвращающий `object`, — результат нужно привести к `Pixel[,]`.

> Если в вашем `Image` есть и другие поля (например, сохранённый заголовок BMP-файла), их тоже нужно передать в копию.

Проверка: напишите код, который доказывает, что `Clone()` работает, а простое присваивание — нет (изменили пиксель в «копии» → посмотрели на оригинал).

Подумайте: если бы внутри `Image` лежал `PixelClass[,]`, была бы копия после `_pixels.Clone()` независимой? (Это контрольный вопрос.)

Часть 4. Фильтры

Все фильтры — статические методы в статическом классе `Filters` (файл `Filters.cs`).

Фильтр 1. Негатив (инверсия)

```
public static void Invert(Image image)
```

Пройдите по всем пикселям. Работайте **через индексатор**: получили копию пикселя → изменили копию → записали пиксель обратно (`image[x, y] = p;`). Без последнего шага ничего не изменится — убедитесь в этом сами.

Для инверсии канала используйте побитовое НЕ: `p.R = (byte)~p.R;`. Почему нужно приведение к `byte` и почему результат равен `255 - p.R`, смотрите в справочном материале («Побитовые операции»).

Фильтр 2. Оттенки серого

```
public static void Grayscale(Image image)
```

Используйте то, что уже написали в `Pixel`: новый пиксель — это `Pixel.Gray(старый.Brightness)`. Тело цикла должно занять одну строку.

Фильтр 3. Яркость (через ссылку)

1. **Публичный метод-оркестратор:** `public static void AdjustBrightness(Image image, int value)` — проходит по всем пикселям, получает **ссылку** на пиксель (`ref Pixel p = ref image.GetPixel(x, y);`) и передаёт её во вспомогательный метод.
2. **Приватный метод-исполнитель:** `private static void AdjustPixelBrightness(ref Pixel p, int value)` — прибавляет `value` к каждому каналу, обрезая результат через `Pixel.Clamp`. О циклах и изображениях он ничего не знает.

Мини-эксперимент: временно уберите `ref` из сигнатуры исполнителя (и из вызова). Скомпилируется ли программа? Изменится ли картинка? Запишите в таблицу

наблюдений и верните `ref` на место.

Фильтр 4. Размытие (Box Blur) и ловушка с копией

`public static void BoxBlur(Image image)`

Для каждого пикселя `(x, y)` (кроме крайних) новый цвет — среднее по квадрату 3×3 вокруг него (9 пикселей, каждый канал отдельно). Читать соседей нужно из **неизменённого** изображения, а писать — в `image`.

Сделайте **две** версии:

1. `BoxBlurWrong(Image image)` — источник получен так: `Image source = image;`
2. `BoxBlur(Image image)` — источник получен так: `Image source = image.Clone();`

Напишите вспомогательный метод `public static int CountDifferent(Image a, Image b)`, который считает, сколько пикселей в двух изображениях **не совпадают** (используйте оператор `!=` из Эксперимента 4).

Загрузите одну и ту же картинку **два раза** (два разных объекта), примените к ним разные версии размытия и выведите результат `CountDifferent`. Объясните, откуда разница.

Часть 5. Сборка в `Main`

Программа должна принимать аргументы командной строки (как это делается — «Считывание из аргументов командной строки» в справочном материале):

```
laba4.exe experiments input.bmp
laba4.exe input.bmp output.bmp invert
laba4.exe input.bmp output.bmp grayscale
laba4.exe input.bmp output.bmp brightness 40
laba4.exe input.bmp output.bmp blur
laba4.exe input.bmp blurcheck
```

- `experiments input.bmp` — запускает все эксперименты из Части 2 по очереди. Картинка нужна для Эксперимента 5 (метод принимает `Image`).
- `blurcheck` — сравнивает две версии размытия и выводит число отличающихся пикселей.
- Без аргументов — выводит подсказку по формату запуска.

Дополнительные задания

Бонус 1. Стеганография: прячем текст в картинке

Напишите два метода:

- `public static void HideText(Image image, string text)` — прячет строку в изображении.
- `public static string ExtractText(Image image)` — достаёт её обратно.

Идея: у каждого канала `R`, `G`, `B` младший бит почти не влияет на цвет — глазу разница в 1 единицу яркости незаметна. Значит, в младший бит каждого канала можно записать один бит своих данных.

1. Переведите строку в байты: `byte[] data = System.Text.Encoding.UTF8.GetBytes(text);`
2. Сначала запишите **длину** сообщения (например, 32 бита `int`), затем сами байты — по одному биту в каждый канал очередного пикселя.
3. Запись бита: `channel = (byte)((channel & 0b1111_1110) | bit);`
4. Чтение бита: `int bit = channel & 1;`
5. Не забудьте проверить, что сообщение помещается в картинку (`Width * Height * 3` бит).

Проверка: спрятали текст → сохранили → загрузили файл заново → достали текст → совпадает.

Бонус 2. Выбор фильтра через делегаты

См. файл «Лабораторная работа 4 - Доп. с делегатами (респект и уважение)».